MindMate: A Local-First Android Study Companion for Task Planning, Scheduling, and Focus Tracking

Amir Nabil
2023170103
Egypt

shimaa shaaban
2023170306
Egypt

jasmine fouad
2023170158
Egypt

pola karmy
2022170098
Egypt

Mohamed tahseen
2023170485
Egypt

Mohamed khaled
2023170485
Egypt

Abstract

Students juggle several subjects, assignment deadlines, and study sessions, yet most general-purpose to-do applications do not model academic work: they have no notion of a subject, a study block, or a focus session. We present *MindMate*, a Kotlin Android application that brings subject management, prioritized task tracking, a daily and weekly study schedule, a Pomodoro focus timer, deadline reminders, and a progress dashboard into a single app. MindMate follows a local-first design: every change is written to a Room database first, so the app remains fully usable offline, and the interface always observes local data. The app is structured with MVVM and the Repository pattern, which keeps UI, business rules, and persistence separate and makes each module independently testable. This abstract describes the problem, the design, the Phase 1 feature set, and the planned evaluation.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**;
• **Applied computing** → *Education*.

Keywords

Android, Kotlin, Room, MVVM, study planner, Pomodoro, local-first

1 Introduction

Academic life requires continuous planning: which subject needs attention first, what is due this week, and when there is time to study. Students commonly struggle to organize subjects and assignments, remember deadlines, divide time between courses, stay focused, and see how much they have actually accomplished. Generic note and task applications can store lists, but they leave students to impose academic structure themselves.

MindMate addresses this gap with a mobile-first planner designed around students' workflows. Its objectives are to (i) organize subjects and tasks, (ii) make study planning simple through daily and weekly schedules, (iii) encourage focused study with a Pomodoro timer, (iv) remind users of upcoming sessions and deadlines, (v) track completed work and study time, and (vi) keep core data available without a network connection.

2 System Design

MindMate is written in Kotlin for Android (minimum API 26). The user interface uses XML layouts with ViewBinding and Material Components, with a single-activity design and AndroidX Navigation between feature fragments. Presentation logic follows **MVVM**: fragments render state exposed by ViewModels and forward user actions, and they never access the database directly. ViewModels communicate with **repositories**, which are the only classes that coordinate data sources.

Persistence uses **Room**. Subjects, tasks, study sessions, and focus sessions are stored as separate entities that carry an owner identifier, so that records are scoped to the signed-in user. Tasks and study sessions reference a subject through a foreign key, and queries return reactive streams so that screens update automatically when data changes. User identity is handled by Firebase Authentication; passwords are never stored in the application database. Asynchronous work uses Kotlin Coroutines, and reminders use WorkManager and notification channels.

3 Phase 1 Features

Phase 1 is graded on four project requirements: use of key Android concepts, UI/UX quality, clean Kotlin code, and persistent data with correct CRUD operations. Table 1 shows how MindMate addresses each one, and the feature modules that implement them follow.

Table 1: Phase 1 requirements and how MindMate meets them.

Requirement	MindMate implementation
Key Android concepts	Explicit intents (notification opens the related screen) and implicit intents (share a task, add a session to the calendar); AndroidX Navigation; RecyclerView; ViewModel with coroutines; WorkManager and notification channels.
UI/UX quality	Material Components, consistent shared styles, bottom navigation, inline form validation, and loading, empty, and error states; layouts adapt to different screen sizes.
Code quality	`val` over `var`, lambdas and higher-order functions, safe nullability, and resource-based strings, colors, and dimensions.
Persistent data	Room database with four entities and DAOs; create, read, update, and delete for subjects, tasks, and study sessions; data survives app restarts.

- **Authentication and profile:** registration, login, logout, persistent sessions, and editable profile information.

- **Subjects (CRUD):** create, view, edit, and delete subjects with a name, optional description, and display color; deletion is confirmed first.
- **Tasks (CRUD):** title, subject, priority (low, medium, high), optional deadline, and completion state, with filtering by status, subject, or priority.
- **Study schedule (CRUD):** study sessions with start and end times, validated time ranges, status (scheduled, completed, cancelled), and daily and weekly views.
- **Dashboard:** today's tasks, upcoming deadlines, today's sessions, and a completed-task summary read from Room.
- **Pomodoro timer:** start, pause, resume, and reset for 25-minute focus periods with 5- and 15-minute breaks; completed focus sessions are stored.
- **Reminders and Intents:** notifications for sessions and deadlines, plus sharing a task and adding a session to the device calendar.

4 Engineering Practices

The team of six works with a Git and GitHub workflow: feature branches, pull requests with at least one review, small focused commits, and issue tracking per feature. Each module owner writes unit tests for business rules such as input validation, time-range checks, and progress calculations, and DAO tests for queries.

5 Evaluation Plan and Future Work

Phase 1 is evaluated by functional testing of the main user journey (register, add a subject, create tasks and sessions, run a focus session, review progress), by verifying that data persists across restarts and works offline, and by checking that one user cannot see another user's data. We do not claim measured learning outcomes at this stage; a usability study with students is left to later work.

Planned extensions include background synchronization of pending local changes to Cloud Firestore with WorkManager, retry and conflict handling, richer statistics, and, in a later release, AI-assisted study planning.

6 Conclusion

MindMate combines subject-aware task management, scheduling, focus tracking, and reminders in a reliable, offline-capable Android application built on a clean MVVM and Repository architecture. Its modular structure lets a team deliver a complete first phase while leaving a clear path to cloud synchronization and intelligent planning.